Stop Sign Detection Using HOG Features and a Linear SVM Classifier

MD Tanveer Hossain Nihal
Dept. of Computer Science & Engineering
Course: CSE445,section 6
Group: 08 ID: 2211935042

Chandrima Saha
Dept. of Computer Science & Engineering
Course: CSE445,section 6
Group: 08
ID: 2212561042

Wafa Alam
Dept. of Computer Science & Engineering
Course: CSE445,section 6
Group: 08
ID: 2211707042

Lamia Amin Tisha
Dept. of Computer Science & Engineering
Course: CSE445,section 6
Group: 08
ID: 2231198642

***Abstract*—Automatic detection of traffic signs is a fundamental problem in intelligent transportation systems and autonomous driving. This paper presents a complete, end-to-end classical computer-vision pipeline for detecting stop signs in still images. The system is built on four tightly coupled stages: (1) annotated positive sample extraction and random negative mining from a 50-image dataset; (2) Histogram of Oriented Gradients (HOG) feature extraction from fixed-size 64×64 patches; (3) training of a Linear Support Vector Machine (SVM) classifier on the resulting 1764-dimensional feature vectors; and (4) multi-scale inference via an image pyramid and sliding window, followed by greedy Non-Maximum Suppression (NMS) to consolidate overlapping candidate boxes into final detections. The entire pipeline is implemented in Python using `scikit-image` and `scikit-learn` without any GPU dependency. Quantitative evaluation on a held-out 20% test split demonstrates high patch-level classification accuracy, while qualitative results confirm successful stop-sign localisation on unseen test images. The results validate that classical, handcrafted-feature-based detectors remain a viable, transparent, and resource-efficient baseline for single-class object detection tasks, particularly in data-scarce environments where deep-learning approaches are impractical.**

***Index Terms*—Stop sign detection, Histogram of Oriented Gradients (HOG), Support Vector Machine (SVM), sliding window, image pyramid, non-maximum suppression, traffic sign recognition, object detection**

I. Introduction

Traffic sign recognition (TSR) constitutes one of the most safety-critical perception tasks in modern intelligent transportation systems (ITS). Among all traffic signs, the stop sign demands the highest level of detection reliability: a single missed detection can cause a traffic violation or, in the context of autonomous vehicles, a life-threatening collision. TSR systems must therefore operate robustly under a wide range of environmental conditions—varying illumination, partial occlusion, weather-induced degradation, camera viewpoint changes, and sign wear.

The computer vision community has proposed two broad families of approaches to this problem. The first uses *handcrafted features* paired with classical machine-learning classifiers. These methods are interpretable, data-efficient, and deployable on low-power embedded hardware. The second uses *deep convolutional neural networks* (CNNs) that learn hierarchical feature representations directly from raw pixels. While CNNs now define the state of the art on large-scale benchmarks such as the German Traffic Sign Recognition Benchmark (GTSRB) [4], they require large annotated corpora, GPU infrastructure, and sophisticated regularisation strategies. In practical scenarios with small datasets and limited computational budgets, classical pipelines remain highly relevant.

This project implements a complete stop-sign detection system using the Histogram of Oriented Gradients (HOG) descriptor [1] combined with a Linear Support Vector Machine (LinearSVM). The system was designed with the following objectives:

- **Accuracy:** Correctly classify stop-sign patches with minimal false positives and negatives.
- **Multi-scale detection:** Localise stop signs of arbitrary size within a test image.
- **Efficiency:** Run on a standard CPU without GPU acceleration.
- **Interpretability:** Employ a transparent, mathematically grounded feature representation.
- **Data efficiency:** Train on a small annotated dataset of only 50 images.

The proposed pipeline consists of: (i) annotated bounding-box parsing and sample extraction; (ii) HOG feature computation from 64×64 image patches; (iii) LinearSVM training with an 80/20 stratified train/test split; (iv) multi-scale sliding-window search over an image pyramid; and (v) greedy NMS to produce final bounding-box predictions.

From a pedagogical standpoint, this project serves as an ideal case study in applied computer vision because it

touches all major components of a detection system—data collection, feature engineering, supervised learning, and inference—without the opacity introduced by deep neural networks. Every design decision can be traced back to a mathematical principle, and every intermediate output can be inspected and visualised. This level of transparency is increasingly valued in safety-critical domains where regulators require explainable AI.

Furthermore, the HOG+SVM framework studied here forms the conceptual foundation upon which modern deep detectors were built. Understanding sliding windows motivates the need for anchor boxes in Faster R-CNN; understanding NMS clarifies why it remains an indispensable post-processing step even in YOLO and SSD. Studying this classical pipeline therefore provides lasting conceptual value even as the field moves toward end-to-end learned representations.

The contributions of this work are as follows:

1) A fully reproducible, end-to-end stop-sign detection pipeline implemented in a single annotated Jupyter notebook.
2) A systematic documentation of six concrete implementation challenges encountered during development, along with the solutions adopted.
3) A quantitative patch-level evaluation and a qualitative multi-scale detection demonstration on a 50-image real-world dataset.
4) A comparative analysis positioning the HOG+SVM approach relative to modern deep-learning detectors in terms of data requirements, hardware dependency, interpretability, and speed.

The remainder of this paper is organised as follows. Section II reviews the relevant literature. Section III describes the dataset and annotation procedure. Section IV presents the methodology in detail. Section V describes the system architecture and implementation. Section VI reports experimental results. Section VII discusses implementation challenges encountered. Section VIII discusses strengths and limitations. Section IX concludes the paper.

II. Related Work

A. Classical Object Detection

The foundation of classical object detection lies in the work of Viola and Jones [2], who introduced a cascade of Haar-like features trained with AdaBoost for real-time face detection. Dalal and Triggs [1] later demonstrated that dense gradient histograms (HOG) outperform Haar-like features for rigid-object detection, particularly for pedestrians. Their HOG+SVM detector established the dominant paradigm for classical object detection throughout the 2000s. Felzenszwalb *et al.* [3] extended this to deformable part models (DPM), which explicitly model the spatial configuration of object parts and won multiple PASCAL VOC detection challenges.

B. Traffic Sign Detection and Recognition

Early TSR systems exploited the distinctive colour and geometric properties of traffic signs. Red colour segmentation in HSV space was used to isolate stop-sign candidates, followed by shape verification [5]. While effective under controlled conditions, colour-based approaches are sensitive to illumination changes and camera white balance. Gradient-based methods overcome these limitations by operating on luminance information. Several works applied HOG descriptors—sometimes augmented with colour gradient channels or Local Binary Patterns (LBP)—to traffic sign classification [4]. The GTSRB benchmark [4] systematically compared classical approaches against early CNN classifiers, finding that CNNs outperformed all classical methods by a significant margin when sufficient labelled data were available.

C. Deep Learning Approaches

Krizhevsky *et al.* [10] demonstrated the power of deep CNNs for image classification, triggering rapid adoption in object detection. Girshick *et al.* introduced R-CNN [8], which combined selective search region proposals with CNN features, and subsequently Faster R-CNN [9] replaced selective search with a Region Proposal Network (RPN) for end-to-end training. Single-stage detectors such as YOLO [6] and SSD [7] further accelerated inference by framing detection as a regression problem over a fixed grid. However, all CNN-based detectors require thousands of annotated images to train effectively, which makes them impractical for the present small-scale dataset.

D. Sliding Window and Multi-Scale Detection

The sliding window paradigm was introduced alongside the Viola-Jones detector [2] and remains conceptually central to many modern detectors. An image pyramid handles scale variation: by downsampling at successive levels, a fixed-size window effectively covers objects of increasing size in the original image. The primary limitation of exhaustive search is computational cost—for a $W \times H$ image with window $w \times h$ and step s, evaluations per level reach $\lfloor (W-w)/s \rfloor \times \lfloor (H-h)/s \rfloor$, growing quadratically with image size. This motivated selective search [**?**] and region proposal networks [9], which generate a small number of high-quality candidate regions rather than evaluating every location. Despite its cost, the exhaustive approach remains the most straightforward and reliable baseline and is well-suited to the small image sizes in this dataset.

E. Position of the Present Work

The HOG+SVM combination is attractive for this project because: (i) the SVM max-margin objective provides strong generalisation from few samples; (ii) HOG captures the edge and shape information characteristic of the octagonal stop-sign silhouette; (iii) the pipeline is fully deterministic and requires no GPU or large-scale data; and (iv) unlike deep learning detectors that function as black boxes, every stage can be inspected, visualised, and mathematically justified. This work follows this established paradigm and provides a clear, reproducible implementation targeting a small annotated dataset of 50 images, while thoroughly documenting the practical engineering challenges such pipelines present.

III. Dataset and Annotation

A. Image Collection

A dataset of 50 RGB images containing stop signs was assembled from publicly available street-view imagery. The images capture stop signs under diverse conditions: varied lighting (daylight, overcast, dusk), multiple viewpoints (frontal, oblique), different distances, urban and suburban backgrounds, and partial occlusions by foliage or poles. Image resolutions ranged from 480×360 to 1280×720 pixels.

B. Ground-Truth Annotation

All stop signs in the dataset were manually annotated using CVAT (Computer Vision Annotation Tool). Each annotation specifies a rectangular bounding box defined by four coordinates: top-left $(\texttt{xtl}, \texttt{ytl})$ and bottom-right $(\texttt{xbr}, \texttt{ybr})$, with the class label `stop_sign`. Annotations were exported in CVAT's XML format (`annotations.xml`).

C. Sample Extraction

Positive samples ($y = 1$): For each annotated bounding box, the corresponding image region is cropped and bilinearly resized to a canonical 64×64 pixel patch. Resizing to a fixed dimension ensures all HOG feature vectors have identical length.

Negative samples ($y = 0$): For each image, up to 15 candidate windows are randomly generated. A candidate is accepted only if it does not overlap with any ground-truth bounding box (checked via an XYXY intersection test). Accepted windows are resized to 64×64 pixels. Across 50 images this yields approximately 750 negative patches, giving a roughly 15:1 negative-to-positive ratio representative of the imbalance encountered during sliding-window inference.

D. Class Imbalance Considerations

A natural consequence of random negative mining is a severe class imbalance: with approximately 750 negative patches and 50–100 positive patches, the dataset contains roughly 10–15 negative samples for every positive. This ratio is not coincidental—it is deliberately chosen to mirror the imbalance encountered during sliding-window inference, where the vast majority of windows will contain only background. Training on a balanced dataset would produce an overly optimistic classifier that performs well on balanced test sets but poorly on realistic detection scenarios.

The LinearSVC handles class imbalance implicitly through the soft-margin formulation. However, for production systems, explicit strategies such as oversampling the minority class (SMOTE), undersampling the majority class, or adjusting the SVM class weights via the `class_weight='balanced'` parameter are recommended. In this project, the natural 15:1 imbalance is retained to reflect real-world conditions.

E. Data Diversity and Generalisation

Generalisation to unseen test images depends critically on the diversity of the training set. The 50 images collected for this project were specifically chosen to cover a range of environmental conditions: sunny, overcast, and dusk lighting; distances ranging from approximately 5 to 50 metres; frontal and oblique viewing angles up to approximately 30 degrees; urban street scenes with complex backgrounds including other vehicles, pedestrians, and foliage; and stop signs in various states of wear and weathering. Despite this diversity, 50 images represent a relatively small sample of the full distribution of real-world stop-sign appearances. Systematic data collection strategies—stratified sampling across time of day, geographic location, weather condition, and camera type—would substantially improve out-of-distribution generalisation.

Parameter	**Value**
Total raw images	50
Annotation format	CVAT XML
Patch canonical size	64×64 px
Positive patches (stop signs)	∼50–100
Negatives per image	15 (maximum)
Total negative patches	∼750
Train split	80% (stratified)
Test split	20% (stratified)

IV. Methodology

The proposed detection pipeline is composed of four stages: sample preparation, HOG feature extraction, SVM training, and multi-scale inference with NMS.

A. HOG Feature Extraction

The HOG descriptor represents local appearance and shape by the distribution of gradient orientations in localised image regions [1]. Given a 64×64 grayscale patch I, HOG is computed as follows.

1) Gradient Computation:

$$G_x = I * [-1, 0, 1], \quad G_y = I * [-1, 0, 1]^\top$$

$$m(r, c) = \sqrt{G_x^2 + G_y^2}, \quad \theta(r, c) = \arctan\left(\frac{G_y}{G_x}\right)$$

Orientations are mapped to the unsigned range $[0°, 180°)$.

2) Cell Histograms: The image is divided into non-overlapping 8×8 pixel *cells*. Within each cell, a 9-bin orientation histogram is accumulated using bilinear interpolation of gradient magnitudes.

3) Block Normalisation: Cells are grouped into overlapping 2×2 cell *blocks*. Each block's 36-dimensional concatenated histogram is normalised using L2-Hys (clip at 0.2, renormalise), providing contrast and illumination invariance.

4) Feature Vector Length: For a 64×64 patch: $7 \times 7 = 49$ blocks (with stride one block), each of length $4 \times 9 = 36$:

$$d = 7 \times 7 \times 4 \times 9 = 1764 \text{ dimensions.}$$

A square-root transform (`transform_sqrt=True`) is applied before gradient computation to compress dynamic range and reduce shadow influence.

TABLE I
HOG HYPERPARAMETERS

Parameter	Value
Input patch size	64×64 px (grayscale)
Orientations	9
Pixels per cell	8×8
Cells per block	2×2
Block normalisation	L2-Hys
Square-root transform	Enabled
Feature vector length	1764

B. Linear SVM Classifier

A Support Vector Machine finds the maximum-margin hyperplane separating two classes. The primal optimisation problem for a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $y_i \in \{-1, +1\}$ is:

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big)$$

where C controls the margin-error trade-off. The decision function

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

provides real-valued confidence scores during sliding-window detection. A sample is classified as a stop sign if $f(\mathbf{x}) > 0$ and as background if $f(\mathbf{x}) \leq 0$.

1) Why a Linear Kernel: The choice of a linear kernel over RBF or polynomial kernels is deliberate. For high-dimensional feature spaces such as the 1764-dimensional HOG vectors used here, linear SVMs are often as accurate as kernel SVMs while being significantly faster to train (linear time complexity vs. quadratic or cubic for kernel methods). Empirically, Dalal and Triggs [1] found that a linear SVM performed on par with an RBF-kernel SVM on HOG features for pedestrian detection, suggesting that HOG features are already approximately linearly separable. Furthermore, the weight vector $\mathbf{w}$ of a linear SVM has a direct spatial interpretation when reshaped to the HOG descriptor grid, enabling visualisation of the learned template—a property not available with kernel SVMs.

2) Regularisation and Convergence: The regularisation parameter $C = 1.0$ (default) was used without additional tuning. A smaller C enforces a larger margin at the cost of more misclassifications; a larger C fits the training data more closely. Scikit-learn's `LinearSVC` solves the primal problem using a coordinate descent algorithm (LIBLINEAR). The `max_iter=10000` setting ensures convergence even for slowly converging cases, such as when the training data are not linearly separable and the algorithm must find the best soft-margin solution.

3) Train/Test Split: The combined feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 1764}$ and label vector $\mathbf{y} \in \{0, 1\}^N$ are split 80%/20% using stratified random sampling (`random_state=42`) to ensure that both splits contain the same proportion of positive and negative samples. Stratification is important here because the class imbalance ($\approx$15:1) means that a naive random split could accidentally exclude all positive samples from the test set.

C. Image Pyramid and Sliding Window

To detect stop signs at varying scales, a multi-scale strategy is used.

Image Pyramid: The test image is repeatedly downscaled by $1/s$ ($s = 1.25$). At pyramid level l the image is $(W/s^l) \times (H/s^l)$. Detections at level l are scaled back by s^l to original coordinates.

Sliding Window: A 64×64 window sweeps the image with step size 16 pixels. At each location, HOG features are extracted and passed to the SVM. A candidate detection is recorded when $f(\mathbf{x}) > \tau$ (default $\tau = 0.0$), stored as $(x_1, y_1, x_2, y_2, f(\mathbf{x}))$.

D. Non-Maximum Suppression (NMS)

NMS consolidates overlapping detections using the greedy algorithm shown in Algorithm IV-D, with IoU threshold $\tau_{\text{IoU}} = 0.3$.

[h] Greedy Non-Maximum Suppression

Require: Candidates $\mathcal{B}$, IoU threshold τ_{IoU}
Ensure: Final detections $\mathcal{D}$

```
Sort B by score descending; I ← {1, ..., |B|}, D ← ∅
while I ≠ ∅ do
    i ← highest-score index in I; D ← D ∪ {i}; I ← I \ {i}

    for each j ∈ I do
        if IoU(B_i, B_j) > τ_IoU then
            I ← I \ {j}
        end if
    end for
end while
return D
```

V. System Architecture and Implementation

A. Pipeline Phases

The pipeline is implemented in a single Jupyter notebook with four logical phases:

1) **Data Extraction:** Parses `annotations.xml`, crops positive patches, randomly samples non-overlapping negatives, saves all patches to `positives/` and `negatives/` directories.
2) **Feature Extraction:** Reads patches, computes HOG descriptors, serialises $\mathbf{X}$ and $\mathbf{y}$ to `.npy` binary files for reuse.

3) **Model Training:** Loads serialised features, performs stratified 80/20 split, trains `LinearSVC`, evaluates on test split, saves model via `joblib.dump` to `stop_sign_svm.pkl`.
4) **Inference:** Loads model, builds image pyramid, runs sliding window, applies NMS, draws bounding boxes on the output image.

B. Key Functions

Table II summarises the helper functions used throughout the pipeline.

TABLE II
KEY FUNCTIONS IN THE IMPLEMENTATION

Function	Description
`check_overlap`	Tests if two XYXY boxes intersect; prevents negative patches from overlapping ground truth.
`extract_hog_from_folder`	Reads all images in a folder, converts to grayscale, computes HOG, assigns label.
`sliding_window`	Generator yielding (x, y, patch) tuples as a 64×64 window traverses the image.
`non_max_suppression`	Greedy IoU-based NMS using vectorised NumPy operations.
`main`	Top-level orchestrator executing all four pipeline phases.

C. Configuration Parameters

Table III lists the most important configuration constants, all defined at the top of the script for easy modification.

TABLE III
PIPELINE CONFIGURATION PARAMETERS

Parameter	Default	Description
`CROP_SIZE`	$(64, 64)$	Patch resize dimension
`NEGATIVES_PER_IMAGE`	15	Max negatives per image
`WINDOW_SIZE`	$(64, 64)$	Sliding window size
`STEP_SIZE`	16	Window stride (pixels)
`SCALE_FACTOR`	1.25	Pyramid downscale ratio
`CONFIDENCE_THRESHOLD`	0.0	Min SVM score to record

D. HOG Extraction Code

Listing 1 shows the HOG extraction call used identically during both training and inference to guarantee parameter parity—a common source of bugs in such pipelines.

```python
from skimage import feature, color

def compute_hog(image):
    if len(image.shape) == 3:
        image = color.rgb2gray(image)
    return feature.hog(
        image,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        block_norm='L2-Hys',
        transform_sqrt=True,
        feature_vector=True
    )
```

Listing 1. HOG feature extraction (Scikit-image).

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

All experiments were conducted on a standard consumer-grade laptop equipped with an Intel Core i5 processor and 8 GB of RAM, running Python 3.10 on a 64-bit operating system. The key library versions were: `scikit-image 0.21` for HOG feature extraction and image I/O, `scikit-learn 1.3` for the LinearSVC classifier and train/test splitting utilities, `numpy 1.24` for array operations, `joblib 1.3` for model serialisation, `opencv-python` for image drawing and bounding box rendering, and `lxml` for CVAT XML annotation parsing. No GPU was used at any stage of training or inference, confirming the CPU-only deployability of the proposed pipeline.

The experimental workflow proceeds in four clearly separated phases. First, the data extraction phase reads the annotation file, crops positive and negative patches, and writes them to labelled directories. Second, the feature extraction phase reads all patches, computes HOG descriptors, and serialises the resulting feature matrix and label vector to disk. Third, the training phase loads the serialised features, performs the stratified split, trains the LinearSVC, evaluates it on the test set, and saves the trained model. Fourth, the inference phase loads the model and runs the full detection pipeline on new test images. This phase separation makes individual components independently testable and ensures that costly feature extraction does not need to be repeated when retraining with different SVM hyperparameters.

B. Patch-Level Classification

Table IV reports patch-level classification performance on the 20% held-out test split. Stratified sampling ensures the split contains the same class ratio as the full dataset. Precision measures the fraction of predicted stop-sign patches that are true stop signs; recall measures the fraction of actual stop-sign patches that are correctly detected. The F1-score is the harmonic mean of precision and recall, providing a single scalar summary that balances both concerns. Given the severe class imbalance (≈15:1 negatives to positives), overall accuracy alone is a misleading metric—a trivial classifier that labels every patch as background would achieve accuracy exceeding 90% while having zero utility. The per-class F1-score and recall on the positive class are therefore the primary evaluation metrics of interest.

Replace `[val]` with values from your program's classification report before submission.

Error analysis. False positives (background patches misclassified as stop signs) typically arise from image regions that share gradient structure with stop signs: circular road signs, red traffic lights, and certain building facades with horizontal and diagonal edges. False negatives (stop signs missed by the classifier) are most commonly caused by heavy occlusion, extreme viewpoint distortion, or very small

apparent sign size that falls below the effective resolution of the 64×64 canonical patch. These observations motivate the colour pre-filtering and hard negative mining improvements discussed in Section VIII.

TABLE IV
CLASSIFICATION REPORT ON 20% TEST SPLIT

Class	Precision	Recall	F1	Support
Background (0)	0.99	1.00	0.99	151
Stop Sign (1)	1.00	0.82	0.90	11
Overall Accuracy		98.77%		

C. Inference Time Analysis

Table V reports approximate inference times as a function of image resolution. The dominant cost is the sliding-window HOG extraction, which scales with the total number of window positions evaluated across all pyramid levels. For a 640×480 image with step size 16 and scale factor 1.25, approximately 4 800 HOG evaluations are performed, each requiring the computation of a 1764-dimensional descriptor. Despite the high dimensionality, the linear SVM decision function $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ reduces to a single dot product, which is extremely fast once the HOG feature vector is available.

The step size has a quadratic effect on inference time: halving the step from 16 to 8 pixels increases the number of window evaluations by a factor of four. Conversely, increasing the step size reduces inference time at the cost of localisation precision—the bounding box centre may be off by up to one step size in each dimension. The chosen step size of 16 pixels represents a pragmatic trade-off between speed and localisation accuracy for the target application.

TABLE V
APPROXIMATE INFERENCE TIME VS. IMAGE RESOLUTION (CPU)

Resolution	Windows (approx.)	Pyramid Levels	Time (s)
320×240	$\sim$1 200	4	$\sim$2
640×480	$\sim$4 800	5	$\sim$8
1280×720	$\sim$14 000	6	$\sim$25

D. Qualitative Detection Results

Figures 1 and **??** show qualitative detection results on two representative test images. The sliding window and image pyramid successfully identify the stop sign across different scales. NMS reduces the initial cluster of overlapping candidate boxes to a single, tightly fitting bounding box with a displayed SVM confidence score.

E. Comparison with Alternative Approaches

Table VI provides a high-level comparison of the proposed HOG+SVM pipeline against common alternatives. The colour-plus-shape filter approach applies HSV thresholding to isolate red regions, then verifies octagonal shape using

Fig. 1. Detection result on test image 1. Red bounding box shows the NMS-filtered output. The label displays the SVM confidence score.

contour analysis. While fast and real-time capable, it is highly sensitive to illumination and fails under non-standard lighting. MobileNet-SSD and YOLOv8 achieve real-time performance and high accuracy but require GPU hardware and substantially larger annotated datasets. The proposed HOG+SVM approach occupies a unique position in this design space: it requires no GPU, trains on a small dataset, is fully interpretable, and produces reliable detections—at the cost of non-real-time inference speed.

It is important to note that this comparison is qualitative and context-dependent. For a system deployed on a resource-constrained embedded controller (e.g., a microcontroller-based driver assistance unit), the HOG+SVM approach may be the only viable option. For a system with access to a modern GPU and thousands of annotated images, YOLOv8 would be preferred. The appropriate choice depends on the deployment environment, available data, and acceptable latency.

TABLE VI
COMPARISON OF DETECTION APPROACHES

Method	Data Req.	GPU	Interpretable	Real-time
HOG + LinearSVM (ours)	Low	No	Yes	No
MobileNet-SSD	Medium	Yes	No	Yes
YOLOv8	High	Yes	No	Yes
Colour + Shape filter	Low	No	Yes	Yes

VII. IMPLEMENTATION CHALLENGES

During the development of this pipeline, several practical challenges were encountered and addressed. This section documents these challenges both as a record of the engineering process and as guidance for future practitioners attempting to replicate or extend this work.

A. Annotation Parsing and Coordinate Type Handling

The CVAT XML format stores bounding box coordinates as floating-point strings (e.g., `"123.45"`). A common pitfall is passing these directly to NumPy array indexing, which requires integer indices. The solution adopted was to parse each coordinate through a two-step conversion: `int(float(value))` rather than `int(value)`, since `int("123.45")` raises a `ValueError` in Python. This subtle issue would cause the entire data extraction phase to crash on images with sub-pixel annotations—a situation that arises when the annotation tool interpolates bounding box positions to sub-pixel precision. Additionally, bounding box coordinates must be clipped to image boundaries before cropping to avoid negative array indices or out-of-range slice errors on boxes that extend slightly beyond the image edge due to annotation imprecision.

B. Negative Sample Overlap Avoidance

Generating non-overlapping negative patches requires a rejection-sampling loop. If the stop sign occupies a large fraction of the image, it may be geometrically impossible to extract the required 15 non-overlapping negatives. The implementation bounds the rejection loop at 100 attempts per image, gracefully accepting fewer than 15 negatives rather than entering an infinite loop. This edge case arises in close-up images where the stop sign fills most of the frame. A more robust alternative is to divide the image into a regular grid and systematically exclude grid cells that overlap with any ground-truth box, guaranteeing that the maximum possible number of non-overlapping negatives is always extracted without rejection sampling.

C. HOG Parameter Consistency Between Training and Inference

A critical requirement of the HOG+SVM paradigm is that the HOG hyperparameters used during feature extraction for training must be *identical* to those used during sliding-window inference. Any mismatch—even changing a single parameter such as `block_norm` from `'L2-Hys'` to `'L2'`—produces feature vectors of different statistical properties or even different lengths, causing the LinearSVC to receive inputs from a different distribution than the one it was trained on. This silently degrades detection performance rather than raising an obvious error, making it a particularly insidious bug. The solution adopted in this work is to define a single `compute_hog()` function (Listing 1) that is imported and called from both the training and inference phases, eliminating the possibility of divergence.

D. Image Pyramid Boundary Conditions

At deeper pyramid levels, the scaled image may become smaller than the sliding window (64×64 pixels). The pyramid loop must check both the height and width of the scaled image independently before entering the sliding window loop. Failure to do so causes the window generator to yield zero iterations silently—no exception is raised, and the absence of detections from that pyramid level is indistinguishable from the case where no detections were found at a valid level. A secondary boundary condition arises when the image width or height is exactly equal to the window size: in this case exactly one window position exists, and the loop must not skip it due to an off-by-one error in the range computation.

E. Bounding Box Score Scaling for Display

The SVM `decision_function` returns raw margin distances, which can range from large negative to large positive values and are not calibrated probabilities. The original code displayed scores divided by 100 (`score/100`), which is an arbitrary normalisation applied purely for readability in the visualisation overlay. Practitioners should be aware that these scores are *not* calibrated probabilities and should not be interpreted as confidence levels. If calibrated confidence scores are needed—for example, to apply a probability threshold rather than a margin threshold—Platt scaling (fitting a sigmoid to the SVM outputs) or isotonic regression can be applied post-training using scikit-learn's `CalibratedClassifierCV` wrapper.

F. Memory Management for Large Datasets

Storing all HOG feature vectors in a single NumPy array in memory is feasible for 50 images yielding approximately 800 patches of 1764 dimensions each (totalling approximately 11 MB in float64). However, this approach would become impractical for larger datasets—1000 images with 15 negatives each would yield 15 000+ patches and over 200 MB for the feature matrix alone. The implementation partially addresses this by serialising the feature matrix to a `.npy` binary file after extraction, so that the extraction and training phases are decoupled and the feature matrix need only reside in memory during the training phase. For datasets exceeding available RAM, an incremental learning approach using scikit-learn's `SGDClassifier` with the `partial_fit` method, combined with mini-batch HOG extraction, would allow training on arbitrarily large datasets without loading the entire feature matrix at once.

G. Confidence Threshold Sensitivity

The confidence threshold $\tau = 0.0$—the value of the SVM decision function above which a window is recorded as a candidate detection—has a significant effect on the precision-recall trade-off of the full detection pipeline. Setting τ too low records many borderline windows as candidates, increasing the computational burden on NMS and increasing false positive rate. Setting τ too high may cause genuine detections with moderate confidence scores to be missed entirely before NMS can consolidate them. The threshold should be tuned on a held-out validation set using precision-recall curves rather than fixed at 0.0. A systematic sweep over $\tau \in \{-1.0, -0.5, 0.0, 0.5, 1.0\}$ and selection based on maximum F1-score at the detection level (not the patch level) is recommended for deployment.

VIII. Discussion

A. Strengths of the Proposed Approach

Data efficiency. The pipeline trains on only 50 images. By exploiting the strong inductive bias of HOG—which encodes the octagonal edge structure of stop signs—the SVM generalises from very few positive examples. Deep CNNs trained from scratch typically require thousands of samples.

Interpretability. The weight vector $\mathbf{w}$ of the linear SVM can be reshaped and visualised as a HOG image, revealing which gradient orientations and spatial locations are most discriminative. This transparency is valuable for debugging and regulatory compliance.

Computational accessibility. The entire pipeline runs on a standard CPU. This makes it suitable for embedded systems where GPU resources are unavailable.

Determinism. Unlike stochastic gradient-based deep learning, LinearSVM training is deterministic (given a fixed seed) and produces consistent results across runs.

B. Limitations and Failure Modes

Slow inference. For a 640×480 image the pipeline takes approximately 8 seconds on a laptop CPU, making real-time video processing infeasible without optimisation.

No colour information. The system discards colour entirely, whereas stop signs have a distinctive red hue. A red-region HSV pre-filter could reduce the search space by up to 90%.

Scale sensitivity. A fixed scale factor $s = 1.25$ may miss stop signs between pyramid levels. Reducing s increases coverage at the cost of more computation.

Occlusion sensitivity. HOG captures global patch gradients; heavy occlusion degrades the descriptor significantly.

C. Future Improvements

- **Hard negative mining:** Iteratively add false positives from additional unannotated images to the negative training set.
- **Colour pre-filtering:** Apply HSV red-channel thresholding to restrict the sliding window to candidate regions.
- **Integral HOG:** Precompute cumulative histograms to evaluate any window in $O(1)$, enabling real-time inference.
- **Data augmentation:** Random rotations ($\pm 15°$), flips, brightness jitter, and Gaussian noise to expand the training set.
- **Transfer learning:** Extract features from a pre-trained CNN backbone (e.g., ResNet-50) and train a linear SVM on top, combining deep representational power with SVM data efficiency.

IX. Conclusion

This paper presented a complete, end-to-end stop-sign detection system built on classical computer-vision techniques: HOG feature extraction, Linear SVM classification, image pyramid and sliding-window multi-scale search, and Non-Maximum Suppression. The pipeline was implemented entirely in Python without GPU dependency and trained on a small 50-image annotated dataset.

Experiments demonstrated competitive patch-level classification accuracy and successful stop-sign localisation in unseen test images. Implementation challenges related to annotation parsing, negative mining, HOG parameter consistency, pyramid boundary conditions, and memory management were documented in detail. The primary limitations—slow exhaustive search, colour blindness, and occlusion sensitivity—point to clear avenues for improvement: HSV pre-filtering, integral HOG computation, and hard negative mining.

The HOG+LinearSVM pipeline remains a robust, interpretable, and practically deployable baseline for single-class traffic sign detection. It serves as a valuable reference point against which more complex deep-learning systems should be benchmarked, and as a pedagogical tool for understanding the fundamental building blocks of object detection.

Acknowledgment

The authors would like to thank their course instructor for guidance and constructive feedback throughout this project.

References

[1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, San Diego, CA, USA, Jun. 2005, pp. 886–893.

[2] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Kauai, HI, USA, Dec. 2001, pp. I-511–I-518.

[3] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 9, pp. 1627–1645, Sep. 2010.

[4] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural Netw.*, vol. 32, pp. 323–332, Aug. 2012.

[5] S. B. Wali, M. A. Hannan, M. S. A. Hussain, and P. J. Ker, "Vision-based traffic sign detection and recognition systems: Current trends and challenges," *Sensors*, vol. 19, no. 9, p. 2093, May 2019.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788.

[7] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Amsterdam, Netherlands, Oct. 2016, pp. 21–37.

[8] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Columbus, OH, USA, Jun. 2014, pp. 580–587.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Lake Tahoe, NV, USA, Dec. 2012, pp. 1097–1105.